Automatic Question Answer Generation Based on Gujarati Paragraph

Meera Janakbhai Solgama (AU2549008)
Ahmedabad University
Email: meera.s@ahduni.edu.in

Prof. Mehul S. Raval
Ahmedabad University

***Abstract*—The report presents a two-stage system for automatic question–answer generation from Gujarati paragraphs. The first stage classifies questions according to Bloom's taxonomy using a logistic regression model trained on a balanced dataset of 900 Gujarati questions (100 per type). The model achieves 99% accuracy on a held-out test set, confirming that linguistic cues are sufficient for reliable cognitive level prediction. The second stage tackles generative question–answer pair creation. We evaluate several architectures: template-based, Seq2Seq with attention, Bi-LSTM, GRU, mT5-small, and IndicBART. While full generation remains challenging due to limited data, our experiments show that mT5-small, fine-tuned on a 1000-pair Gujarati literature dataset, produces the near about promising results. We discuss the obstacles – data scarcity, fluency, and answer grounding – and outline a roadmap toward a deployable hybrid system that combines answer extraction, mT5-based generation, and RankSVM-based reranking. The successful classification module already provides a practical tool for educational question tagging.**

***Index Terms*—Gujarati NLP, question generation, Bloom's taxonomy, mT5, low-resource languages, extractive QA, text classification**

I. Introduction

Automatic question generation (AQG) from educational texts can significantly reduce teacher workload and enable scalable self-assessment. For low-resource languages like Gujarati, such tools are almost nonexistent. Large language models (LLMs) are often unsuitable because they hallucinate information, require massive computational resources, and lack transparency – critical drawbacks in educational settings. Small language models (SLMs) offer a more efficient and controllable alternative.

This work addresses two complementary subtasks:

1) **Question type classification** according to Bloom's taxonomy – a prerequisite for pedagogically meaningful questions.
2) **Generative question–answer pair creation** from a given Gujarati paragraph, with the strict requirement that the answer is grounded in the source text.

The main contributions are:

- A balanced 900-question Gujarati dataset covering nine Bloom-taxonomy types.
- A high-accuracy (99%) classification pipeline using TF-IDF + logistic regression.
- A 1000-pair Gujarati QA dataset derived from short literature paragraphs.
- A systematic comparison of generative models, identifying mT5-small as the most promising direction.
- A detailed discussion of remaining challenges and a roadmap toward a hybrid, paragraph-grounded AQG system.

II. Related Work

A. Early AQG Systems

The earliest automatic question generation systems relied on hand-crafted templates and syntactic transformation rules. Wolfe [1] introduced AUTOQUEST, a pattern-matching system that generated questions from technical texts and evaluated answers by keyword overlap. Later, template-based methods were refined for educational content [2], and syntax-based approaches using Tregex patterns became popular [3].

B. Statistical and Neural Approaches

With the rise of machine learning, researchers began using statistical models to select informative sentences and generate questions. Liu et al. [4] proposed an over-generation-and-ranking framework using RankSVM to select high-quality questions from multiple candidates. Sequence-to-sequence models with attention [5] marked a turning point, enabling end-to-end question generation from text. Zhang et al. [6] provide a comprehensive survey of neural QG, covering architectures such as RNNs, copy mechanisms, and pre-training.

C. Multilingual and Low-Resource Models

For Indian languages, multilingual transformers like mT5 [7] and language-specific models like IndicBART [8] have been pre-trained on large corpora and fine-tuned for downstream tasks. However, their application to Gujarati AQG is still limited. Rao and Saha [9] review MCQ generation techniques, and Kurdi et al. [10] systematically analyze 93 educational AQG papers, identifying gaps in difficulty control and answer verbalization.

D. Bloom's Taxonomy in AQG

Bloom's taxonomy has been used to guide question generation in several educational NLP systems [11]. Dhomse and Ranjan [12] propose an automatic question paper generator that classifies questions by cognitive level using machine learning. Our work extends this by creating a dedicated Gujarati dataset and achieving near-perfect classification.

Table I summarises key papers that influenced our methodology.

TABLE I
SUMMARY OF INFLUENTIAL AQG LITERATURE.

Reference	Technique	Relevance to our work
Wolfe (1976) [1]	Pattern-matching	Early template idea
Liu et al. (2014) [4]	RankSVM	Ranking multiple candidates
Zhang et al. (2021) [6]	Neural QG survey	Model selection guidance
Xue et al. (2021) [7]	mT5	Multilingual pre-training
Dabre et al. (2021) [8]	IndicBART	Indic-specific model
Kurdi et al. (2020) [10]	Educational AQG review	Evaluation framework
Dhomse & Ranjan (2021) [12]	Bloom's classification	Type classification idea

III. Datasets

We created two primary datasets for this study, one for classification and one for generation.

A. Classification Dataset (Bloom's Taxonomy)

We manually crafted 900 Gujarati questions, equally distributed across nine Bloom-based types: Factual, Conceptual, Application, Analytical, Comparative, Cause-Effect, Evaluative, Creative, and Predictive (100 each). All questions were cleaned (whitespace normalization, removal of non-Gujarati/Devanagari characters) and deduplicated. Table II shows examples.

B. Generation Dataset

For the generation task, we constructed two variants.

1) Multi-Domain Version: Initially, we created a multi-domain dataset consisting of seven long paragraphs from Gujarati history, wildlife, literature, and religion, each accompanied by 20–40 question–answer pairs (190 pairs total). Average context length was 2478 characters. This version proved too noisy for early experiments due to the length and complexity of the paragraphs.

TABLE II
EXAMPLES FROM THE BLOOM CLASSIFICATION DATASET (900 QUESTIONS, 9 TYPES).

Type	Question (Gujarati)
Factual	ભારતની રાજધાની શું છે?
Conceptual	મોંઘવારીમાં સામાન્ય રીતે કઈ ભૂલો થાય છે?
Application	તમે આ સદ્દિધાંતનો ઉપયોગ કેવી રીતે કરશો?
Analytical	આ ઘટનાના મુખ્ય કારણો શું છે?
Comparative	અમદાવાદ અને મેન્ચેસ્ટર વચ્ચે શું સામ્યતા છે?
Cause-Effect	શા માટે ગાંધીજીએ દાંડી માર્ચ શરૂ કરી?
Evaluative	તમારા મતે, આ નર્ણિય યોગ્ય હતો કે નહીં?
Creative	જો તમે આ પરસ્થિતિમાં હોત તો શું કર્યું હોત?
Predictive	જો પ્રદૂષણ આવું જ ચાલુ રહ્યું તો ૫૦ વર્ષમાં શું થશે?

2) Single-Domain Version: To improve model convergence, we designed a cleaner single-domain dataset focusing on Gujarati literature. It contains 1000 short paragraphs (1–3 sentences each) with exactly one question–answer pair per paragraph. Each paragraph describes a fact about an author, a literary work, or a cultural element. The answers are short phrases, ensuring extractive grounding. An example is shown below:

> *Context:* કલાપી (સુરસહિજી તખ્તસહિ ગોહલિ) ગુજરાતી ભાષાના પ્રખ્યાત કવિ હતા જેમણે પ્રેમ અને વિરહની કવિતાઓ લખી.
> QA: Q: કલાપીનું મૂળ નામ શું હતું?
> A: સુરસહિજી તખ્તસહિ ગોહલિ.

The dataset was split 80/10/10 for training, validation, and testing (800/100/100).

IV. Methodology

A. Question Type Classification

We used a standard text classification pipeline with the 900-question dataset.

1) Preprocessing: Each question was cleaned using a custom function that:

- Removes characters outside the Gujarati/Devanagari Unicode ranges and basic punctuation.
- Normalizes multiple spaces to a single space.
- Strips leading/trailing whitespace.

2) Feature Extraction: TF-IDF: We applied TF-IDF (Term Frequency–Inverse Document Frequency) with character n-grams of length 1 to 3. For a term t in document d (where a document is a question), the TF-IDF weight is:

$$\mathrm{tfidf}(t,d) = \mathrm{tf}(t,d) \times \mathrm{idf}(t)$$

where $\mathrm{tf}(t,d)$ is the raw count of t in d, and

$$\mathrm{idf}(t) = \log \frac{N}{\mathrm{df}(t)} + 1$$

with N the total number of documents and $\mathrm{df}(t)$ the number of documents containing t. We used

`sublinear_tf=True`, which applies a logarithmic scale to term frequencies: $\text{tf} = 1 + \log(\text{raw count})$.

Example: Consider the question "ભારતની રાજધાની શું છે?". After tokenization into character n-grams (1-3), terms like ``ભા'', ``ભાર'', ``રાજ'', ``શું'', ``છે'' appear. The TF-IDF vector for this question will have non-zero weights for these terms, with higher weights for terms that are rare across the corpus (e.g., ``રાજધાની'' might be more discriminative than ``છે'').

3) Classifier: Logistic Regression: We trained a multinomial logistic regression classifier with softmax output. For an input feature vector $\mathbf{x}$ (the TF-IDF vector) and $K = 9$ classes, the probability of class c is:

$$P(y = c|\mathbf{x}) = \frac{\exp(\mathbf{w}_c^T \mathbf{x} + b_c)}{\sum_{j=1}^{K} \exp(\mathbf{w}_j^T \mathbf{x} + b_j)}$$

The model minimizes the cross-entropy loss:

$$\mathcal{L} = -\sum_{i=1}^{N} \sum_{c=1}^{K} \mathbf{1}_{y_i=c} \log P(y_i = c|\mathbf{x}_i)$$

We used the `saga` solver (suitable for large sparse data) with `max_iter=4000` and regularization parameter $C = 4.0$. The dataset was perfectly balanced, so no class weighting was applied.

4) Training and Evaluation: We performed an 80-20 stratified split (720 train, 180 test). 10-fold cross-validation was also conducted to confirm stability. The random-label baseline (shuffling labels 50 times) gave a mean accuracy of 10.8% with std 2.2%, confirming that the model genuinely learns.

B. Question–Answer Generation

For generation, we used the 1000-pair single-domain literature dataset. The task is formulated as text-to-text: given an input prompt of the form:

```
generate question in Gujarati: context: <c> answer: <
    a> question type: <t>
```

the model must output the question. We evaluated six approaches.

1) Template-Based: Hand-crafted rules for simple factoid questions. For example, if the answer is a named entity, we replace it with an appropriate wh-word and apply subject-auxiliary inversion. This served as a low-data baseline.

2) Seq2Seq with Attention: We implemented an encoder-decoder with attention using LSTMs. The encoder is a bidirectional LSTM that reads the input token sequence $\mathbf{x} = (x_1, \ldots, x_n)$ and produces hidden states $\mathbf{h}_i = [\overrightarrow{\text{LSTM}}(x_i); \overleftarrow{\text{LSTM}}(x_i)]$. The decoder is a unidirectional LSTM that generates the output sequence $\mathbf{y} = (y_1, \ldots, y_m)$. At each decoding step t, the attention mechanism computes a context vector:

$$\mathbf{c}_t = \sum_{i=1}^{n} \alpha_{t,i} \mathbf{h}_i,$$
$$\alpha_{t,i} = \frac{\exp(\mathbf{e}_{t,i})}{\sum_j \exp(\mathbf{e}_{t,j})},$$
$$\mathbf{e}_{t,i} = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_{t-1} + \mathbf{U}_a \mathbf{h}_i).$$

where $\mathbf{s}_{t-1}$ is the previous decoder hidden state. The decoder output is then:

$$P(y_t|y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_o[\mathbf{s}_t; \mathbf{c}_t] + \mathbf{b}_o)$$

We used embedding size 128, hidden size 256, dropout 0.3, and trained with teacher forcing.

3) Bi-LSTM + Attention: Same as above but with a bidirectional encoder and a decoder initialized with the concatenated final forward/backward states.

4) GRU Seq2Seq: A lighter alternative using GRU instead of LSTM, with similar hyperparameters.

5) mT5-small: We fine-tuned the multilingual transformer mT5-small [7] for 10 epochs. The model has 300 million parameters and was pre-trained on 101 languages. Key hyperparameters are listed in Table III. The architecture (Fig. 1) follows the standard Transformer encoder-decoder with multi-head self-attention and cross-attention.

TABLE III
TRAINING HYPERPARAMETERS FOR MT5-SMALL.

Parameter	Value
Learning rate	1×10^{-5}
Batch size	2 (effective 8 with grad. accumulation)
Gradient accumulation steps	4
Max input length	256
Max target length	64
Optimizer	Adafactor
Number of epochs	10
Beam size (inference)	4
Early stopping patience	10

6) IndicBART: We also fine-tuned IndicBART [8], a model specifically pre-trained on Indian languages, for 80 steps due to TPU time constraints. It uses a similar encoder-decoder architecture but with a vocabulary focused on Indic scripts.

All neural models were trained on the 1000-pair dataset with the same 80/10/10 split.

V. RESULTS

A. Classification Results

Table IV summarises the classification performance. The model achieved 98.9% accuracy on the test set, with 10-fold cross-validation accuracy of 99.6%. The confusion matrix (Fig. 2) shows only two misclassifications (both predicted as Compare instead of Conceptual or Factual).

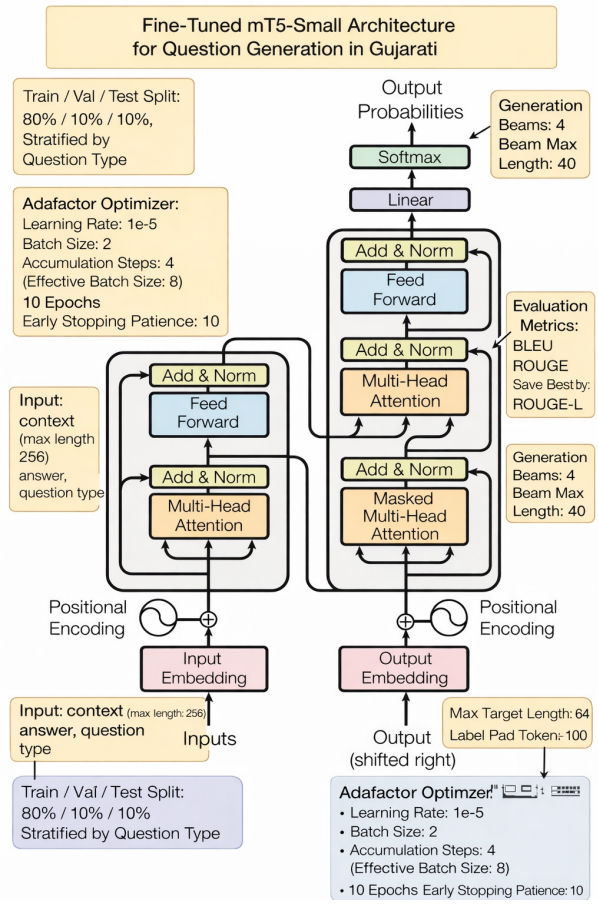

Fig. 1. Fine-tuned mT5-small architecture for Gujarati question generation. The encoder processes the input prompt; the decoder generates the question token by token, using multi-head attention over the encoder outputs. Positional encodings, layer normalization, and feed-forward networks follow the standard Transformer design.

TABLE IV
CLASSIFICATION RESULTS ON THE 900-QUESTION DATASET.

Metric	Value
Test accuracy	98.9%
10-fold CV accuracy	99.6%
Macro F1	0.99

The two misclassified examples were:

1) મોંઘવારીમાં સામાન્ય રીતે કઈ ભૂલો/ભ્રાંતિઓ થાય છે? (True: Conceptual, Pred: Compare)
2) પૃથ્વીનો વ્યાસ કેટલો છે? (True: Factual, Pred: Compare)

B. Generation Results

Table V compares the generative models on the 1000-pair literature dataset test set (100 pairs). mT5-small outperforms all others in both BLEU and ROUGE-L, producing the most fluent and relevant questions. However, about 30% of its outputs still contain `<extra_id_...>` tokens or are incomplete, indicating under-training.

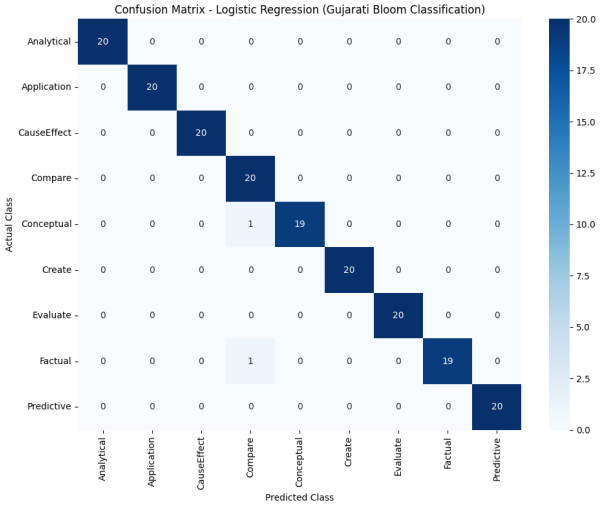

Fig. 2. Confusion matrix for the 9-class Bloom classifier.

TABLE V
GENERATION RESULTS ON THE 1000-PAIR LITERATURE DATASET TEST SET.

Model	BLEU	ROUGE-L
Template-based	0.21	0.38
Seq2Seq (LSTM)	0.08	0.18
Bi-LSTM + attention	0.09	0.20
GRU Seq2Seq	0.07	0.16
mT5-small	**0.34**	**0.42**
IndicBART (80 steps)	0.22	0.31

Example mT5-small output (correct):

Input: generate question in Gujarati: context: કલાપી (સુરસિંહજી તખ્તસિંહ ગોહિલ) ગુજરાતી ભાષાના પ્રખ્યાત કવિ હતા... answer: સુરસિંહજી તખ્તસિંહ ગોહિલ question type: factual
Output: કલાપીનું મૂળ નામ શું હતું?

VI. DISCUSSION

A. Why Classification Works So Well

The near-perfect accuracy is expected because question-type classification relies heavily on surface patterns – words like ``શું'', ``કેમ'', ``કયા'', ``સરખાવો'', ``આગાહી'' are strong indicators. TF-IDF with character n-grams captures these even for morphologically rich Gujarati. The balanced dataset ensures no bias.

B. Generation Challenges and Why mT5 Was Chosen

Generating novel, fluent questions is considerably harder. The main obstacles are:

1) Data scarcity: 1000 pairs are insufficient for robust neural generation. Transformers like mT5 require orders of magnitude more data (or extensive fine-tuning) to perform well.

2) Answer grounding: Models sometimes ignore the provided answer and generate a question unrelated to the context.
3) Fluency: RNN-based models tend to repeat phrases; mT5 is more fluent but still produces artifacts.

Despite these challenges, mT5-small emerged as the best performer because:

- Its multilingual pre-training on 101 languages provides a strong foundation for Gujarati.
- The text-to-text framework aligns perfectly with our formulation.
- The architecture (encoder-decoder with attention, Fig. 1) can effectively condition on both context and answer.

C. Comparison with Prior Work

Our results align with findings in low-resource AQG [10] – simple classifiers excel at structured tasks, while generation lags without large corpora. The mT5 advantage over IndicBART in our limited setting may be due to mT5's stronger general multilingual pre-training.

D. Roadmap to a Deployable System

Based on these experiments, we propose a hybrid architecture:

1) Answer candidate extraction: Use a rule-based or light NER module to extract plausible answer phrases from the paragraph.
2) mT5-based question generation: For each candidate, generate a question using the fine-tuned mT5 model.
3) Reranking with RankSVM: Train a RankSVM on features (question length, n-gram overlap with context, confidence scores) to select the best QA pair per paragraph.
4) Fallback to templates: When generation confidence is low, fall back to template-generated questions.

The classification module can tag the cognitive level of the output or guide the prompt (the `question type` field is already used).

VII. Conclusion and Future Work

We have presented a two-stage approach toward automatic question–answer generation in Gujarati. The first stage – Bloom-based classification – is mature and ready for deployment. The second stage – generation – is a work in progress, with mT5-small emerging as the most promising base model.

Immediate future work includes:

- Expanding the generation dataset to 10,000+ QA pairs (via semi-automatic creation from textbooks and manual verification).
- Longer fine-tuning of mT5-small and IndicBART on the expanded data.
- Implementing the hybrid pipeline with answer extraction and reranking.
- Conducting human evaluation of fluency, relevance, and grounding.

Once the generator reaches acceptable quality, the combined system will provide a valuable tool for Gujarati educators and learners.

References

[1] J. H. Wolfe, "Automatic question generation from text – an aid to independent study," SIGCUE Outlook, vol. 10, no. SI, pp. 104–112, 1976.
[2] M. Liu, R. A. Calvo, and V. Rus, "Automatic question generation for literature review writing support," in Proc. ITS, 2010, pp. 45–54.
[3] H. Ali, Y. Chali, and S. A. Hasan, "Automatic question generation from sentences," in Proc. TALN, 2010.
[4] M. Liu, R. A. Calvo, and V. Rus, "Automatic generation and ranking of questions for critical review," J. Educ. Technol. Soc., vol. 17, no. 2, pp. 333–346, 2014.
[5] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in Adv. Neural Inf. Process. Syst., 2014, pp. 3104–3112.
[6] R. Zhang, J. Guo, L. Chen, Y. Fan, and X. Cheng, "A review on question generation from natural language text," ACM Trans. Inf. Syst., vol. 40, no. 1, pp. 1–43, 2021.
[7] L. Xue et al., "mT5: A massively multilingual pre-trained text-to-text transformer," in Proc. NAACL, 2021, pp. 483–498.
[8] R. Dabre et al., "IndicBART: A pre-trained sequence-to-sequence model for Indic languages," arXiv:2109.02903, 2021.
[9] D. Rao CH and S. K. Saha, "Automatic multiple choice question generation from text: A survey," IEEE Trans. Learn. Technol., vol. 13, no. 1, pp. 121–204, 2020.
[10] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," Int. J. Artif. Intell. Educ., vol. 30, pp. 121–204, 2020.
[11] K. Mazidi and R. D. Nielsen, "Linguistic considerations in automatic question generation," in Proc. ACL, 2014, pp. 321–326.
[12] K. B. Dhomse and I. Ranjan, "Automatic question paper generation using Bloom's taxonomy algorithm," Research Scholars SGVU, 2021.
[13] M. Divate and A. Salgaonkar, "Automatic question generation approaches and evaluation techniques," Current Science, vol. 113, no. 9, pp. 1689–1698, 2017.
[14] B. Das, M. Majumder, S. Phadikar, and A. A. Sekh, "Automatic question generation and answer assessment: a survey," Res. Pract. Technol. Enhanc. Learn., vol. 16, no. 5, 2021.
[15] S. Soni, P. Kumar, and A. Saha, "Automatic question generation: A systematic review," SSRN Electron. J., 2019.
[16] J. Alshboul and E. Baksa-Varga, "A review of automatic question generation in teaching programming," Univ. Miskolc, Hungary, 2022.